

How to Build a One-Person HFT Hedge Fund on Jev That Fires Calibrated Trades on Every Block 24/7

A Field Report on Splitting a Trading System Into Deterministic Computation and Probabilistic Judgment Using the First System One Decision Model

Abstract—Every automated trading system faces one structural conflict at its decision layer. Deterministic rules are fast and reproducible but cannot generalize when the market shifts regime. Large language models generalize but require three to thirty seconds per decision, by which point a block-cadence order book has moved dozens of times. For fifty years this forced a binary choice between fast and blind or smart and late, a gap institutions closed only with field-programmable gate arrays, colocation, and quantitative teams costing millions per desk. On September 15, 2026, TypeSafe AI released Jev, the first System One model, a decision engine that returns typed calibrated probabilities in seventy to five hundred milliseconds at 0.042 dollars per million input tokens with output billed at zero. This paper documents a production architecture in which Jev occupies the probabilistic decision layer of a twenty-four-hour high-frequency market-making loop on block-cadence venues, while deterministic code retains ownership of state computation, thresholds, risk vetoes, and execution, and the surrounding system is constructed with the AgenKit structured agentic harness. The central claim is that the model does not replace the trading system. It replaces the fuzzy judgment layer inside it, and that single substitution places continuous autonomous market making within reach of a solo operator for tens of dollars per month.

Index Terms—Jev, System One model, TypeSafe AI, AgenKit, high-frequency trading, market making, calibrated decisions, RLCD, deterministic probabilistic split, confidence gating, Avellaneda-Stoikov inventory control, fractional Kelly sizing, block-cadence execution, one-person hedge fund, Monad, on-chain order book.

I. INTRODUCTION: THE DECISION LAYER GAP

For fifty years the automated trading stack has been forced into a false choice at its decision layer. A deterministic rule fires in microseconds but is blind to any condition its author did not anticipate. If spread is below a threshold, quote. If drawdown exceeds a limit, stop. Clean, testable, and unable to recognize a regime it was never coded for. A language model recognizes the regime but answers in seconds, and on a three hundred millisecond block that answer is already stale when it arrives.

This paper documents a production system built around a third option released in September 2026. TypeSafe AI, founded by Diogo Almeida, co-inventor of ChatGPT and InstructGPT at OpenAI, emerged from two years of stealth with forty million dollars in funding and a new class of model. It is called a System One model, after Kahneman's fast intuitive cognition. It does not generate text. It evaluates typed questions against a state and returns calibrated probability distributions that surrounding code consumes directly. The model is named Jev.

A. Why Language Model Usage Fails at the Decision Layer

Every solo operator who has attempted to place a frontier language model inside a high-frequency loop has met the same wall. The model decodes its answer token by token, sequentially, bound by memory bandwidth on the accelerator. A single buy or sell classification is generated one character at a time, then the surrounding code must parse it, validate it, and repair malformed output. End to end this consumes three to thirty seconds. On a block-cadence venue producing a new state every three hundred milliseconds, the decision describes a market that no longer exists. The intelligence was present. The latency made it inadmissible.

B. The System One Definition

This paper defines the System One decision layer as the configuration in which a model evaluates a compact deterministic state against a battery of atomic typed questions, returns calibrated probabilities in parallel, and hands those probabilities to surrounding code that owns every threshold, composition rule, and side effect. Under this configuration the model does not trade. It judges. The code decides. This division is the architecture that makes the system testable, auditable, and safe to run unattended.

C. The Rules vs LLM vs Jev Comparison

To frame the contribution, Table I compares the three decision-layer options documented in this paper across the six properties that determine whether a decision engine can sit inside a twenty-four-hour high-frequency market-making loop.

TABLE I
DETERMINISTIC RULES VS LLM VS JEV AT THE DECISION LAYER

Property	Rules	LLM	Jev
Latency	Microseconds	3-30 s	70-500 ms
Generalizes	No	Yes	Yes
Calibrated	No	No	Yes (RLCD)
Cost / decision	Zero	High	Near zero
Failure mode	Blind	Stale	Abstains
Auditable	Yes	No	Yes (typed)

D. What This Paper Delivers

Section II specifies the System One model class, its three primitives, and the RLCD training that makes its confidence meaningful. Section III presents the deterministic-probabilistic architecture and the AgenKit construction methodology. Section IV documents the market state engine. Section V presents the parallel judgment battery and the confidence-gated policy engine. Section VI details the twenty-four-hour market-making loop and the fallback ladder. Section VII presents the risk engine and the calibration framework. Section VIII presents cost analysis, honest scope, and the precise boundary of what Jev replaces.

II. THE SYSTEM ONE MODEL CLASS

A System One model is a distinct class of frontier model that does not generate text. Where a large language model maps a state to a sequence of tokens, a System One model maps a state to a decision distribution. This section specifies the model class, its three primitives, its parallel evaluation property, and the training method that makes its confidence scores usable as sizing inputs.

A. State to Decision, Not State to Text

The interface is two-part. The caller submits a *state*, which is any JSON structure describing the situation to be judged, and a set of typed questions, each with a predefined answer space. The model returns one typed answer per question with an associated probability distribution. There is no prose, no parsing step, and no possibility of a malformed or off-schema response, because the answer space is fixed in advance by the caller. For a trading system this means an order book snapshot enters as state and a calibrated *buy*, *sell*, or *hold* distribution returns as a value the code can branch on directly.

B. The Three Primitives

The model exposes exactly three question types. *Noul* returns a single probability between zero and one, answering whether a statement about the state is true. *Choice* selects one option from a predefined list of up to two hundred fifty-five candidates and returns the full probability distribution across them. *Score* places the state on an ordered rubric of two to ten levels defined by the caller. Any number of these three may be combined in one request. Equations (1) through (3) give the return signature of each primitive, where s denotes the state and q denotes a question.

$$\text{Noul}(s, q) \rightarrow p \in [0, 1] \quad (1)$$

$$\text{Choice}(s, q) \rightarrow \arg\max_i p_i, \quad \{p_1, \dots, p_n\}, \quad n \leq 255 \quad (2)$$

$$\text{Score}(s, q) \rightarrow v \in [\ell_{\min}, \ell_{\max}], \quad \{p_1, \dots, p_k\}, \quad 2 \leq k \leq 10 \quad (3)$$

Each primitive returns not only a chosen value but the full distribution over the answer space and a confidence scalar. The distribution is the object the trading policy consumes. A direction question that returns *up* with probability 0.55 and a direction question that returns *up* with probability 0.91 produce the same $\arg\max$ and radically different position sizes.

C. Parallel Evaluation in One Pass

The defining architectural property is that every question in a request is evaluated in parallel and in isolation against the same state. The model is not autoregressive. It does not decode one answer, append it to context, and condition the next answer on it. It computes all distributions in a single forward pass. The consequence for latency is decisive: a battery of six questions returns in the same seventy to five hundred milliseconds as a single question. Isolation also eliminates context contamination, since no question can influence another. Equation (4) states the latency invariance, where m is the number of questions.

$$T(s, \{q_1, \dots, q_m\}) \approx T(s, q_1), \quad \forall m \leq M \quad (4)$$

Because each additional question adds only the input tokens of the question itself, and output tokens are billed at zero, the marginal cost of asking one more question is negligible. This inverts the prompt-engineering economy of language models, where every additional judgment multiplies latency and cost. Here the correct design is to ask every question that might inform the decision, including those whose answers will be discarded.

D. RLCD: Why the Confidence Is Meaningful

The model is trained with Reinforcement Learning for Calibrated Decisions, a method that optimizes reported probabilities against realized outcomes rather than against human preference. This is the property that separates the model class from a language model emitting token log-probabilities. A language model's stated confidence is not calibrated to outcomes and cannot be used as a sizing input without gross error. Under RLCD, events the model tags at eighty percent occur, in aggregate, approximately eighty percent of the time. Equation (5) states the calibration target, where p is the reported probability and Y is the outcome indicator.

$$P(Y = 1 \mid \hat{p} = p) \approx p, \quad \forall p \in [0, 1] \quad (5)$$

This single guarantee is what licenses fractional Kelly sizing directly from model output, developed in Section V. The context window is thirty-two thousand tokens, sufficient for a dense order book snapshot, inventory state, and recent trade history, and deliberately insufficient for an entire strategy document. The model judges compact state. The code owns everything else. Table II summarizes the three primitives and their trading application.

TABLE II
THE THREE PRIMITIVES AND THEIR TRADING APPLICATION

Primitive	Returns	Range	Trading use
Noul	Probability	0 to 1	Is flow toxic
Choice	Option + dist	≤ 255	Regime, direction
Score	Level + dist	2-10	Setup quality

III. THE DETERMINISTIC-PROBABILISTIC ARCHITECTURE

The central architectural decision of this system is the boundary between what is computed and what is judged. Everything that can be calculated exactly is calculated in deterministic code. Everything that is an inherent judgment is delegated to Jev. This section states the split principle, its rationale, and the construction methodology by which the surrounding system was built.

A. The Split Principle

The model must not own the trading system. It must own selected judgments inside the trading system. Every failed attempt to place an AI model at the center of a trading loop shares one error: it asks the model to trade. The instruction to observe an instrument and decide what to do is unanswerable at any latency because it compresses a dozen independent judgments and a dozen deterministic computations into one opaque request. The correct decomposition assigns four distinct responsibilities in sequence: code calculates the state, Jev interprets the state, code applies policy, and the execution engine places the order.

B. What Stays Deterministic

Every quantity with an exact definition remains in code. Mid price, microprice, spread in basis points, order book imbalance, queue position, realized volatility, inventory, drawdown, and the Avellaneda-Stoikov reservation price are arithmetic, not judgment. Delegating arithmetic to a probabilistic model is both slower and less accurate than computing it directly. This mandate is consistent with the model vendor's own guidance that arithmetic, authorization, thresholds, and side effects must remain in code.

C. What Becomes a Judgment

Every quantity that requires interpretation of ambiguous state becomes a typed question. Whether the current regime is trending or mean-reverting, whether aggressive flow is informed or noise, whether the order book configuration is a high-quality setup, and whether execution has degraded are judgments no closed-form formula resolves. These are the questions Jev answers, each atomic, each composed with explicit weights in code.

D. Atomic Questions Composed in Code

A question that would require extended reasoning or that weighs several independent factors is decomposed into separate questions, each scoped to a single factor, and recombined by the caller's own logic. When a priority shifts, the operator changes a coefficient in code rather than rewriting a prompt. This keeps every individual evaluation reliable and places the composition rule under version control rather than inside an opaque model.

III.E CONSTRUCTING THE SYSTEM WITH AGENKIT

The split principle defines what to build. It does not build it. The deterministic half of the system, comprising the state engine, the policy composition, the risk vetoes, the pricing engine, the execution adapter, and the twenty-four-hour loop, is a full production codebase of tested modules. Hand-wiring these modules, each with its own failure modes and test surface, is the stage at which most solo builds collapse. The system documented in this paper was constructed with AgenKit, available at agenkit.xyz.

AgenKit is a structured agentic engineering harness installed into a coding environment such as Codex CLI or Claude Code. It does not add model capability. It installs a ten-discipline engineering team, each discipline a role-scoped agent, and drives them through six approval-gated phases. The operator approves the specification, the architecture, and the plan before any code is written. Every task is built test-first, reviewed against the specification, and shipped with rollback. The result is that the deterministic scaffolding around Jev is produced with senior-team discipline rather than as an unstructured script.

F. Phase to Module Mapping

Table III maps each of the six AgenKit phases to the concrete trading-system artifact it produced in the construction of this system, establishing the reproducibility path from a written specification to a shipped market-making loop.

TABLE III
AGENKIT PHASE TO TRADING MODULE MAPPING

Phase	Specialist	Artifact produced
Brainstorm	Conductor	System spec
Architecture	Architect	Module boundaries
Plan	Conductor	Test-first tasks
Build	Engineers	State + policy code
Review	Reviewer	Spec-match verdict
Ship	Infra + Security	Loop + kill switch

The division is now complete on both axes. At runtime, Jev owns judgment and code owns computation. At build time, AgenKit owns the engineering discipline and the operator owns the approval gates. Jev decides. AgenKit builds. The following sections document each produced module in turn, beginning with the market state engine.

IV. THE MARKET STATE ENGINE

The quality of every downstream decision is bounded by the quality of the state handed to Jev. The state engine is pure deterministic code that runs on every block and emits a compact numeric snapshot. This section specifies the seven feature families of the snapshot, the three construction rules that keep it valid, and the reference schema.

A. Seven Feature Families

The snapshot computes seven families of feature. *Price* captures mid, microprice, and returns over multiple horizons. *Order book* captures spread, depth across the top levels, imbalance, and queue position. *Flow* captures aggressive buy and sell volume, trade intensity, and cancel intensity. *Volatility* captures realized volatility across two windows and the ratio of current volatility to its twenty-four-hour regime. *Cross-venue* captures divergence against a reference venue, basis, and funding. *Portfolio* captures inventory, unrealized profit, drawdown, and position age. *Execution health* captures recent fill ratio, reject rate, observed slippage, and the decision latency of the last ten calls.

B. Three Construction Rules

Three rules keep the snapshot valid. First, the snapshot remains under four hundred tokens, because latency scales with input size and the caller pays per input token, so the state must be dense and numeric with no prose. Second, timestamp discipline is absolute: every field uses only information timestamped strictly before the decision instant, since any look-ahead in the state engine invalidates both the live decision and the calibration record. Third, every snapshot is logged with the decision it produced and the outcome that followed, because that triple is the sole raw material for the calibration analysis of Section VII.

C. The Causal Timestamp Constraint

The timestamp constraint deserves formal statement because it is the most common source of invalid backtests. Let a decision be taken at time t . Every feature in the snapshot must be a function only of information with timestamp strictly less than t . Equation (6) states the constraint, where x_t is the snapshot and $I_{<t}$ is the information set available strictly before t .

$$x_t = f(I_{<t}), \quad I_{<t} = \{\text{observations } \tau : \tau < t\} \quad (6)$$

D. Reference Schema

The following schema is the reference snapshot passed as state to Jev on each block. All values are numeric. The structure is flat for compactness and every field is computed deterministically from the order book and trade feed.

```
snapshot = {
  # price
  'mid': 3.4127,
  'microprice': 3.4131,
  'ret_1m': 0.0021,
  'ret_5m': -0.0008,

  # order book
  'spread_bps': 3.5,
  'imbalance': 0.71,
  'depth_3': 42150.0,
  'queue_pos': 4,

  # flow
  'aggr_buy': 0.63,
  'aggr_sell': 0.37,
  'trade_intensity': 8.2,
  'cancel_intensity': 5.1,

  # volatility
  'rvol_5m': 0.034,
  'rvol_30m': 0.041,
  'vol_regime': 1.18,

  # portfolio
  'inventory': -120,
  'upnl': 14.7,
  'drawdown': 0.038,
  'pos_age_s': 96,

  # execution health
  'fill_ratio': 0.82,
  'slippage_bps': 1.4,
  'latency_ms': 81
}
```

This snapshot is approximately two hundred forty tokens serialized, well within the four-hundred-token budget, and is regenerated on every block event. It is the sole input Jev receives. No raw order book, no trade history, and no strategy description is passed, because the state engine has already reduced the market to the minimal sufficient statistic the judgment layer requires.

E. The State Engine as a Bottleneck Guard

Because the state engine is deterministic and fully under version control, it is also the natural location for the first line of defense. A field that cannot be computed, a feed that has gone stale, or a value outside its plausible range is caught here, before any question reaches Jev. A malformed snapshot is never sent. This guard converts a class of silent live failures into explicit, logged, pre-decision rejections, and it is the first of several layers that keep the system alive without supervision.

V. THE JUDGMENT BATTERY AND POLICY ENGINE

This section presents the core runtime of the system: the battery of typed questions posed to Jev on every block, and the deterministic policy engine that composes their probabilities into an action. The battery is a single request. The policy engine is pure code. Together they convert a state snapshot into a quote decision in one model call.

A. The Six-Question Battery

A single question is a demonstration. A production system poses a full battery, exploiting the latency invariance of Section II: six questions return in the same time as one. The battery decomposes the trade into six atomic judgments, each scoped to one factor, mirroring how a market maker reasons about the same order book. The following call is the runtime core.

```
resp = client.system_one(
    state=snapshot,
    questions={
        'regime': Choice(
            criteria={'trending': None,
                    'mean_reverting': None,
                    'high_vol': None, 'crisis': None}),
        'direction': Choice(
            criteria={'up': None, 'down': None,
                    'neutral': None}),
        'toxic_flow': Noul(
            'Is aggressive flow informed?'),
        'liquidity_stressed': Noul(
            'Book thinner than 24h norm?'),
        'quote_env': Score(
            legend={'0': 'No', '1': 'Marginal',
                    '2': 'Standard', '3': 'Excellent'}),
        'inventory_pressure': Score(
            legend={'0': 'None', '1': 'Mild',
                    '2': 'Skew', '3': 'Reduce now'}),
    })
```

Six judgments return in one call at a marginal cost near one hundred-thousandth of a dollar per block. Two Choice questions classify regime and direction, two Noul questions gate on toxicity and liquidity stress, and two Score questions rate the quoting environment and inventory urgency. Every answer arrives with a full probability distribution and a confidence scalar.

B. The Confidence-Gated Policy Engine

The policy engine is deterministic and owned entirely by the operator. It applies vetoes first, then gates each action on the confidence appropriate to the cost of being wrong. The thresholds live in code and are changed by editing a coefficient, never by reprompting the model.

```
def compose(a, snap, lim):
    if snap['drawdown'] > lim.max_dd:
        return KILL
    if a['toxic_flow'].noul > 0.6:
        return PULL_QUOTES
    if a['liquidity_stressed'].noul > 0.7:
        return WIDEN

    q = a['quote_env']
    if q.score >= 2 and q.confidence > 0.80:
        skew = skew_of(a['inventory_pressure'])
        return quote_both(skew)
    if q.score >= 1:
        return quote_wide()
    return STAND_DOWN
```

Three principles govern the engine. Vetoes precede signals, so a drawdown breach or a toxicity read overrides any positive judgment. Thresholds are per-action, not global, because a wrong stand-down costs nothing while a wrong full-size quote costs capital, so their gates differ. And position size derives from calibrated probability through fractional Kelly, licensed only because RLCD makes the probability meaningful.

C. Fractional Kelly From Calibrated Probability

When the policy engine elects to size a directional position, the fraction of the size limit committed is a capped fraction of the Kelly criterion, computed directly from Jev's reported probability. Equation (7) gives the sizing rule, where p is the calibrated probability of the chosen direction and c is the fractional cap, set to one quarter in this deployment.

$$f = c \cdot \max(0, 2p - 1), \quad c = 0.25 \quad (7)$$

This computation is defensible only under a calibrated model. Applied to the token log-probabilities of a language model, which are not calibrated to outcomes, the same formula produces systematic over-sizing. Table IV summarizes the six questions, their primitive types, and the policy action each drives.

TABLE IV
THE SIX-QUESTION BATTERY AND ITS POLICY ROLE

Question	Type	Drives
Regime	Choice	Strategy mode
Direction	Choice	Quote skew
Toxic flow	Noul	Pull veto
Liquidity	Noul	Widen veto
Quote env	Score	Quote gate
Inventory	Score	Skew size

VI. THE 24/7 MARKET-MAKING LOOP

This section assembles the components of the preceding sections into the continuous loop that justifies the twenty-four-hour claim. The deterministic half of market making is decades-old mathematics that remains in code. Jev answers only the question the mathematics cannot: whether the current environment is one worth quoting into at all.

A. Deterministic Pricing

The quoting engine computes the Avellaneda-Stoikov reservation price and optimal half-spread on every block. These are closed-form and deterministic. Equation (8) gives the reservation price and equation (9) the optimal half-spread, where s is mid, q is inventory, γ is risk aversion, σ is volatility, $T - t$ is the horizon, and κ is the order arrival rate.

$$r = s - q\gamma\sigma^2(T - t) \quad (8)$$

$$\delta = \gamma\sigma^2(T - t) + \frac{2}{\gamma} \ln\left(1 + \frac{\gamma}{\kappa}\right) \quad (9)$$

Jev does not compute these values and never should. Its role is upstream: the quote environment score of Section V decides whether the engine quotes at all, quotes wide, or stands down, and the inventory pressure score sets the skew. The mathematics prices; the judgment gates.

B. The Block Loop

On each block event the loop reads the order book, computes the snapshot, fires the battery, composes the action, prices the quotes, checks hard limits, and posts. The following is the loop skeleton.

```
async def on_block(evt):
    book = read_l2(evt)
    snap = state_engine(book, pos)
    if not valid(snap):
        return hold()

    ans = jev_battery(snap)
    act = compose(ans, snap, limits)
    if act is KILL:
        return flatten()

    r, d = avellaneda_stoikov(snap)
    if not risk_ok(act, pos, limits):
        return hold()

    cancel_prev()
    post_only(act, r, d)
    log(snap, ans, act)
```

C. The Block Deadline Rule

A decision that has not returned before the next block deadline is discarded, and the loop holds rather than posting a quote computed on stale state. This single rule is the difference between a system that degrades gracefully under latency spikes and one that accumulates adverse fills against a market it can no longer see. The seventy to five hundred millisecond decision budget fits inside a three hundred millisecond block only when the deadline is enforced and co-location reduces the round trip.

D. The Fallback Ladder

Autonomy over twenty-four hours requires a defined response to every degraded state, not merely the healthy one. The fallback ladder specifies five operating states in descending order of model availability, each mapped to a deterministic action that executes without the model. Table V gives the ladder.

TABLE V
THE FALLBACK LADDER

Model state	Deterministic action
Healthy, high confidence	Normal operation
Healthy, low confidence	Reduce size
Late past deadline	Hold, no repost
Unavailable	Fallback rules only
Hard limit breached	Kill, flatten, alert

The ladder is the operational definition of autonomy. A system with it continues to behave correctly when the model is slow, uncertain, or absent, because every degraded state has a deterministic answer that does not depend on the model. A system without it is autonomous only until its first network partition. The economic viability of the loop is examined next, since a loop that survives technically can still lose money to transaction costs.

E. The Gas Constraint

On a block-cadence venue, naive cancellation and replacement of quotes on every block incurs a transaction cost on every block. When that cost exceeds the captured spread, the loop is technically alive and economically dead. The loop must therefore widen quotes, extend resting time, or require a directional edge from the battery sufficient to clear the cost floor. This constraint is quantified in the cost analysis of Section VIII.

VII. RISK ENGINE AND CALIBRATION

Two layers stand between the judgment battery and unsupervised operation with real capital. The risk engine enforces deterministic limits that the model can never override. The calibration framework verifies that the model's confidence means on this venue what the policy engine assumes it means. This section specifies both.

A. Deterministic Risk Vetoes

The risk engine is hard-coded, checked before every order, and holds zero negotiation authority. Its limits cover maximum position, maximum leverage, maximum order size, maximum daily loss, maximum drawdown, maximum inventory age, maximum stale-data age, maximum API error rate, and maximum decision latency. Every limit is verifiable by an independent measurement, not by the model's own claim that it ran. Jev supplies soft judgments that adjust size and posture; the risk engine supplies hard vetoes that halt. When they conflict, the veto wins without exception.

B. Why the Model Never Holds a Veto

A calibrated judgment is a probability, and a probability is not a guarantee. A toxicity read of 0.7 is correct to widen or pull quotes, but it is not a substitute for a hard position limit, because the three-in-ten case still occurs. The risk engine exists precisely for the tail the judgment does not cover. Delegating a hard limit to a soft judgment reintroduces the failure mode the deterministic layer was built to eliminate.

C. The Calibration Requirement

RLCD calibrates the model against the training distribution of its vendor. The distribution of a specific venue, instrument, and question phrasing is not that distribution. The policy engine sizes positions on the assumption that a reported probability of p corresponds to an outcome frequency of p , and that assumption must be verified on the operator's own logged data before capital is sized on it. The state-decision-outcome triples logged since Section IV are the material for this verification.

D. Calibration Metrics

Calibration is measured on the logged triples with two standard metrics. The Brier score is the mean squared error between reported probability and realized outcome. The Expected Calibration Error is the weighted average gap between confidence and accuracy across probability bins. Equations (10) and (11) give both, where p_i is the reported probability, y_i the outcome, B_b the set of predictions in bin b , and n the sample count.

$$BS = \frac{1}{n} \sum_i (p_i - y_i)^2 \quad (10)$$

$$ECE = \sum_b \frac{|B_b|}{n} \cdot |acc(B_b) - conf(B_b)| \quad (11)$$

The following procedure computes the reliability of the direction question over the logged history and returns the per-bin gap that drives any correction.

```
def reliability(logs, bins=10):
    gap = []
    for bin in range(bins):
        lo, hi = bin/bins, (bin+1)/bins
        s = [l for l in logs
              if lo <= l.p < hi]
        if not s:
            continue
        conf = mean(l.p for l in s)
        acc = mean(l.y for l in s)
        gap.append((bin, conf, acc,
                    acc - conf))
    return gap
```

When the reliability curve departs from the diagonal, a monotonic recalibration such as Platt scaling is fitted in the policy layer and applied to the reported probability before sizing. Calibration is verified per question, since a model may be well calibrated on direction and poorly calibrated on regime.

E. The Four-Baseline Evaluation

The system is not validated by asking whether Jev predicts price. It is validated by comparing the full policy against three alternatives on identical data, costs, and limits. Table VI gives the four baselines and the metric that distinguishes them.

TABLE VI
THE FOUR-BASELINE EVALUATION

Baseline	Isolates
Deterministic rules	No generalization
Frontier LLM layer	Latency cost
Jev layer	Calibration value
Jev + confidence gate	Abstention value

VIII. COST, SCOPE, AND CONCLUSION

This section presents the operating cost of the system, the precise boundary of what Jev does and does not replace, and the conclusion. The cost figures are the actual monthly operating cost of the deployment. The scope statement is the honest contract by which the system should be judged.

A. Operating Cost

The system runs on three cost components. Jev inference, at 0.042 dollars per million input tokens with output billed at zero, costs approximately ten to twenty-five dollars per month for one fully loaded six-question battery per block, twenty-four hours per day. The AgenKit engineering harness that builds and maintains the deterministic code is fifteen dollars per month. Market data on a block-cadence venue is the venue's own feed. The total operating cost is on the order of tens of dollars per month, against an institutional market-making desk whose field-programmable gate arrays, colocation, and quantitative team represent millions of dollars in annual fixed cost. Table VII gives the comparison.

TABLE VII
OPERATING COST VS INSTITUTIONAL DESK

Component	One-person	Institutional
Decision layer	\$10-25/mo	Quant team
Build layer	\$15/mo	Eng team
Infrastructure	Venue feed	FPGA + colo
Order of cost	Tens/mo	Millions/yr

B. What Jev Replaces

Jev replaces the fuzzy judgment layer of the trading system: the hand-written regime classifiers, the heuristic toxicity rules, the manually tuned setup filters, and the language-model classification calls that were previously the only way to generalize. In each case a brittle rule or a slow generative call is replaced by a fast calibrated typed judgment.

C. What Jev Does Not Replace

Jev does not generate text, design a strategy, or trade. It does not chain dependent judgments in one call, since questions are evaluated in isolation. It does not compete in the microsecond lane, where field-programmable gate arrays and colocation remain mandatory. It does not replace market-data infrastructure, numerical computation, exchange connectivity, or the deterministic risk engine. And it does not create edge: it makes decisions fast and cheap, but a strategy without an edge loses faster, not less.

IX. CONCLUSION

The automated trading stack has for fifty years faced a false choice at its decision layer between deterministic rules that are fast and blind and language models that are intelligent and late. The System One model class resolves this choice. Jev returns calibrated typed decisions in seventy to five hundred milliseconds, fast enough to sit inside a block-cadence loop and calibrated enough to size positions directly from its output.

The system that wins with it is not the one that asks the model to trade. It is the one that computes everything computable in code, poses one compact state to a full battery of atomic questions, gates every action on calibrated confidence, prices with deterministic mathematics, and holds hard risk vetoes with absolute authority. Jev supplies the judgment. The surrounding code, built with AgenKit and documented across this paper, supplies the system. Together they place a twenty-four-hour autonomous market maker within reach of a solo operator at an operating cost of tens of dollars per month.

The institutional moat at the microsecond frontier did not fall. It remains defended by physics and capital. What fell is the moat at the block-cadence frontier, where the binding constraint was never the mathematics of market making but the cost and latency of the judgment layer between the order book and the quote. That constraint is now priced in tens of dollars per month. The desk is one person.

REFERENCES

- [1] TypeSafe AI, "Introducing System One Models and Jev," Official release announcement, Sep. 15, 2026.
- [2] D. Almeida, "System One models: calibrated decisions via RLCD," TypeSafe AI Technical Report, 2026.
- [3] TypeSafe AI, "Jev Documentation: Primitives, State, and Confidence," docs.typesafe.ai, 2026.
- [4] M. Avellaneda and S. Stoikov, "High-frequency trading in a limit order book," *Quantitative Finance*, vol. 8, no. 3, pp. 217-224, 2008.
- [5] J. L. Kelly, "A new interpretation of information rate," *Bell System Technical Journal*, vol. 35, no. 4, pp. 917-926, 1956.
- [6] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1-3, 1950.
- [7] J. Platt, "Probabilistic outputs for support vector machines," *Advances in Large Margin Classifiers*, pp. 61-74, 1999.
- [8] D. Kahneman, *Thinking, Fast and Slow*. New York: Farrar, Straus and Giroux, 2011.
- [9] J. Watts, "jev-trader: a Jev-powered market maker on Monad," GitHub repository, 2026.
- [10] AgenKit, "Structured agentic engineering harness," agenkit.xyz, 2026.